

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América



FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

---

## **Semana 1 - POO**

---

**Curso:**

Programación de Computadoras II

**Docente:**

Javier Elmer Cabrera Díaz

**Autor:**

Mamani Huahualuque, Airton Abedal

**Lima, 31 de Agosto del 2026**

# Índice

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                     | <b>2</b>  |
| <b>2. Desarrollo</b>                                       | <b>2</b>  |
| 2.1. Sumario de cuatro herramientas CASE . . . . .         | 2         |
| 2.1.1. <i>StarUML</i> . . . . .                            | 2         |
| 2.1.2. <i>Visual Paradigm</i> . . . . .                    | 3         |
| 2.1.3. <i>Enterprise Architect</i> . . . . .               | 3         |
| 2.1.4. <i>ArgoUML</i> . . . . .                            | 3         |
| 2.1.5. <i>Comparación general</i> . . . . .                | 4         |
| 2.2. Struct vs. Class . . . . .                            | 4         |
| 2.3. Ampliación de la prueba en el método main() . . . . . | 5         |
| 2.4. Tipos de datos en Java . . . . .                      | 7         |
| 2.5. ¿Por qué se obtiene $a+b = -294967296$ ? . . . .      | 7         |
| 2.5.1. <i>Representación conceptual</i> . . . . .          | 8         |
| 2.5.2. <i>Solución utilizando long</i> . . . . .           | 9         |
| 2.6. Relación con el caso de la clase TV . . . . .         | 9         |
| <b>3. Conclusión</b>                                       | <b>11</b> |
| <b>4. Referencias</b>                                      | <b>11</b> |

# 1 Introducción

La presente tarea corresponde a la primera semana del curso de Programación 2 y tiene como eje la transición desde la programación estructurada hacia la Programación Orientada a Objetos (POO).

De acuerdo con el material de clase, Programación 1 se relaciona con la programación estructurada, incluyendo secuencia, alternativa, estructuras repetitivas, modularidad y archivos. Por otro lado, Programación 2 introduce la Programación Orientada a Objetos, cuyo enfoque permite representar elementos del mundo real mediante objetos relacionados.

Los principales conceptos de la POO que se presentan en esta etapa son la abstracción, el encapsulamiento, la herencia y el polimorfismo.

A partir de esta introducción, el presente trabajo desarrolla cinco actividades planteadas por el docente: un sumario de cuatro herramientas CASE relacionadas con la ingeniería de software; una comparación entre `struct` y `class`; la ampliación de la prueba realizada en el método `main()` del ejemplo de la clase; una revisión de los tipos de datos de Java; y la explicación del resultado negativo obtenido al sumar dos valores enteros de gran magnitud.

El objetivo es comprender no solamente la sintaxis utilizada en Java, sino también los conceptos que justifican la organización de un programa mediante clases y objetos.

En particular, el caso de la clase TV permite relacionar atributos, métodos, creación de instancias, validación de datos y encapsulamiento con una situación concreta.

## 2 Desarrollo

### 2.1 Sumario de cuatro herramientas CASE

CASE (*Computer-Aided Software Engineering*) es el término utilizado para referirse a herramientas informáticas que apoyan distintas actividades de la ingeniería de software.

Dependiendo de la herramienta, pueden facilitar el análisis, modelado, diseño, documentación, generación de código, gestión del proyecto u otras actividades relacionadas con el ciclo de desarrollo de software.

#### 2.1.1 StarUML

StarUML es una herramienta orientada al modelado de sistemas mediante UML. Permite elaborar diferentes tipos de diagramas, como diagramas de clases, casos de uso, secuencia, actividad y componentes.

Es especialmente útil para representar visualmente la estructura y el comportamiento de un sistema antes de implementar el código.

**Aplicación en POO:**

En el contexto de Programación Orientada a Objetos, puede utilizarse para diseñar clases, identificar atributos y métodos y representar las relaciones existentes entre diferentes clases.

**2.1.2 Visual Paradigm**

Visual Paradigm es una herramienta de ingeniería de software que ofrece capacidades de modelado UML y apoyo a diferentes actividades del desarrollo.

Permite trabajar con distintos tipos de diagramas y documentar modelos del sistema.

**Aplicación en POO:**

Puede emplearse para pasar de una representación conceptual del sistema a una especificación más detallada de las clases y sus relaciones.

**2.1.3 Enterprise Architect**

Enterprise Architect es una herramienta de modelado y diseño de sistemas utilizada para trabajar con UML y otros modelos relacionados con el desarrollo de software.

Está orientada a proyectos donde se necesita documentar y administrar modelos de manera amplia.

**Aplicación en POO:**

En un proyecto de POO permite representar clases, interfaces, herencia, asociaciones y otras relaciones que posteriormente pueden ser implementadas en Java u otro lenguaje.

**2.1.4 ArgoUML**

ArgoUML es una herramienta de modelado UML de código abierto, utilizada para crear modelos y diagramas de software.

Su enfoque está relacionado principalmente con el modelado y la representación de estructuras de software.

**Aplicación en POO:**

Puede servir como recurso académico para practicar la identificación de clases, atributos, operaciones y relaciones antes de escribir la implementación.

### 2.1.5 Comparación general

Tabla 1: Comparación de herramientas CASE

| Herramienta          | Enfoque principal       | Utilidad en POO                     | Tipo                               |
|----------------------|-------------------------|-------------------------------------|------------------------------------|
| StarUML              | Modelado UML            | Diseño de clases y relaciones       | Herramienta CASE/modelado          |
| Visual Paradigm      | Modelado y desarrollo   | Modelos UML y documentación         | Herramienta CASE                   |
| Enterprise Architect | Modelado y arquitectura | Clases, interfaces y relaciones     | Herramienta CASE                   |
| ArgoUML              | Modelado UML            | Práctica de estructuras y diagramas | Herramienta CASE de código abierto |

## 2.2 Struct vs. Class

La comparación entre `struct` y `class` depende del lenguaje de programación. En el contexto de la tarea, la comparación es especialmente útil para diferenciar una estructura de datos de una clase utilizada en Programación Orientada a Objetos.

En lenguajes como C++, `struct` y `class` pueden utilizarse para definir tipos que agrupan datos y funciones.

Una diferencia fundamental es el nivel de acceso predeterminado: en una `struct` de C++, los miembros son públicos por defecto, mientras que en una `class` son privados por defecto.

También es posible utilizar herencia con ambos mecanismos.

En Java no existe la palabra reservada `struct`. Para crear tipos definidos por el programador se utiliza principalmente `class`, además de interfaces, enumeraciones y otros mecanismos.

Por ello, cuando el curso trabaja Programación Orientada a Objetos en Java, la clase es el mecanismo central para representar objetos.

Tabla 2: Comparación entre struct y class

| Aspecto                      | struct (C++)                     | class (Java/C++)                                      |
|------------------------------|----------------------------------|-------------------------------------------------------|
| Existencia en Java           | No existe como palabra reservada | Sí, es el mecanismo principal para definir clases     |
| Acceso predeterminado en C++ | public                           | private                                               |
| Uso típico                   | Agrupar datos y funciones        | Modelar entidades/objetos y encapsular comportamiento |
| POO                          | Puede participar en POO en C++   | Es fundamental para POO en Java                       |
| Instancias                   | Se pueden crear objetos en C++   | Se crean objetos mediante new en Java                 |

En conclusión, para el curso de Programación 2 en Java, la comparación sirve principalmente para comprender que Java trabaja con clases como base de la abstracción y la creación de objetos, mientras que struct es un concepto propio de otros lenguajes como C/C++.

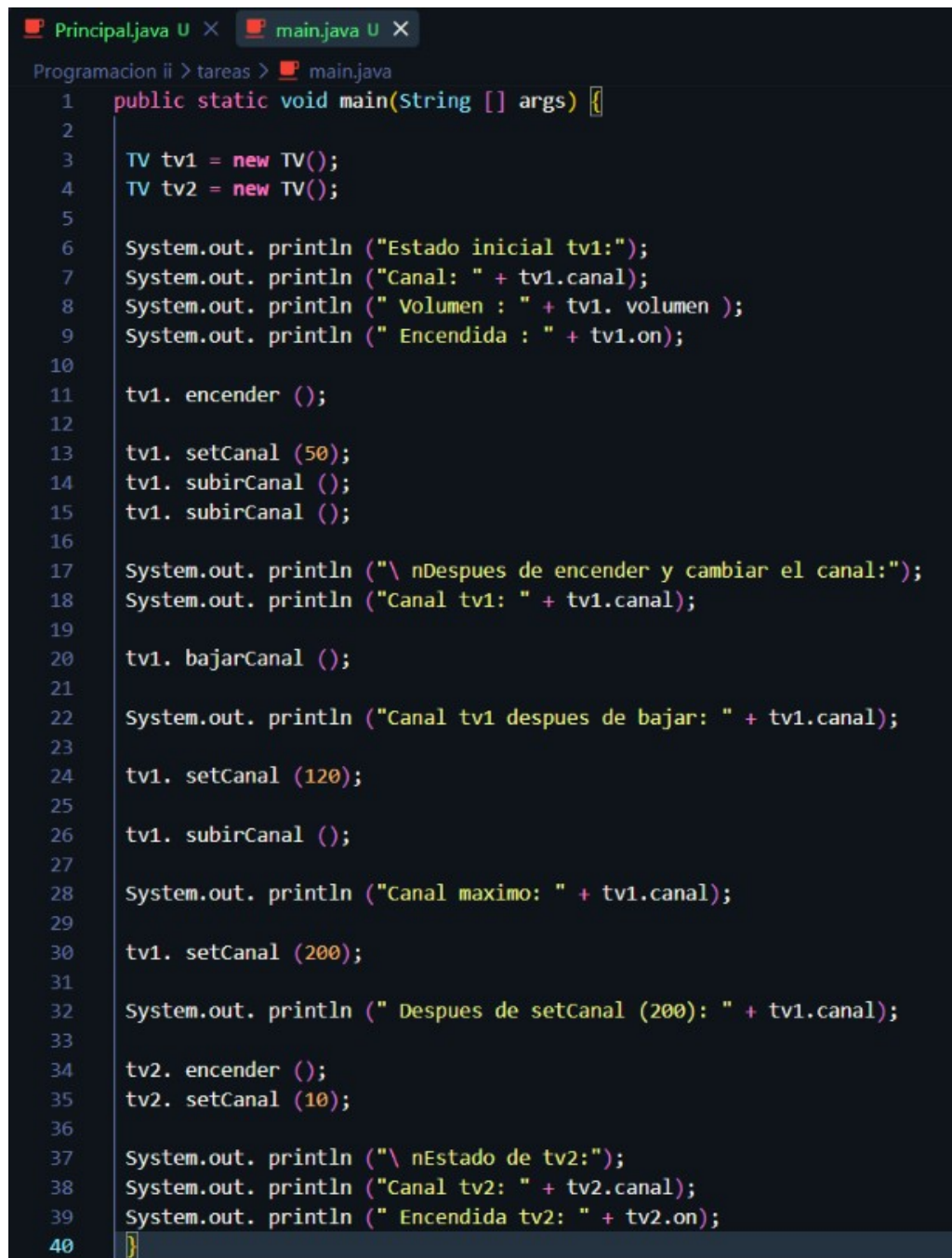
## 2.3 Ampliación de la prueba en el método main()

El ejemplo de la clase utiliza una clase TV con tres atributos: canal, volumen y on.

Además, define operaciones para encender y apagar el televisor, subir y bajar el canal y establecer un canal válido.

El programa crea dos instancias, tv1 y tv2, lo que permite demostrar que dos objetos pertenecen a la misma clase, pero mantienen su propio estado.

Una ampliación útil del método main() consiste en probar más operaciones y comprobar los límites establecidos por la clase.



```
Principal.java U x main.java U x
Programacion ii > tareas > main.java
1 public static void main(String [] args) {
2
3     TV tv1 = new TV();
4     TV tv2 = new TV();
5
6     System.out. println ("Estado inicial tv1:");
7     System.out. println ("Canal: " + tv1.canal);
8     System.out. println (" Volumen : " + tv1. volumen );
9     System.out. println (" Encendida : " + tv1.on);
10
11    tv1. encender ();
12
13    tv1. setCanal (50);
14    tv1. subirCanal ();
15    tv1. subirCanal ();
16
17    System.out. println ("\ nDespues de encender y cambiar el canal:");
18    System.out. println ("Canal tv1: " + tv1.canal);
19
20    tv1. bajarCanal ();
21
22    System.out. println ("Canal tv1 despues de bajar: " + tv1.canal);
23
24    tv1. setCanal (120);
25
26    tv1. subirCanal ();
27
28    System.out. println ("Canal maximo: " + tv1.canal);
29
30    tv1. setCanal (200);
31
32    System.out. println (" Despues de setCanal (200): " + tv1.canal);
33
34    tv2. encender ();
35    tv2. setCanal (10);
36
37    System.out. println ("\ nEstado de tv2:");
38    System.out. println ("Canal tv2: " + tv2.canal);
39    System.out. println (" Encendida tv2: " + tv2.on);
40 }
```

**Figura 1**

*Ampliación del método main() para validar el comportamiento y límites de la clase TV*

*Nota.* Captura de pantalla de la implementación del método main() en el entorno de desarrollo, donde se evalúa la independencia de estados entre objetos y los límites de los métodos de control. Elaboración propia.

Esta prueba permite observar una idea central de POO: cada objeto posee su propio estado.

Aunque tv1 y tv2 fueron creados a partir de la misma clase TV, cambiar el canal de tv1 no cambia el canal de tv2.

## 2.4 Tipos de datos en Java

Java posee tipos de datos primitivos y tipos de referencia.

Los tipos primitivos representan valores básicos, mientras que los tipos de referencia permiten trabajar con objetos, arreglos, cadenas y otros tipos definidos por el programador.

Tabla 3: Principales tipos de datos en Java

| Tipo    | Categoría  | Ejemplo                         | Descripción                        |
|---------|------------|---------------------------------|------------------------------------|
| byte    | Entero     | <code>byte x = 100;</code>      | Entero pequeño con signo           |
| short   | Entero     | <code>short x = 1000;</code>    | Entero con rango mayor que byte    |
| int     | Entero     | <code>int x = 200;</code>       | Entero de 32 bits                  |
| long    | Entero     | <code>long x = 200L;</code>     | Entero de 64 bits                  |
| float   | Decimal    | <code>float x = 2.5f;</code>    | Decimal de precisión simple        |
| double  | Decimal    | <code>double x = 2.5;</code>    | Decimal de doble precisión         |
| char    | Carácter   | <code>char c = 'A';</code>      | Representa un carácter Unicode     |
| boolean | Lógico     | <code>boolean ok = true;</code> | Representa true o false            |
| String  | Referencia | <code>String s = "Hola";</code> | Cadena de caracteres; es una clase |

Para el ejemplo de la clase, el tipo `int` es especialmente importante porque su capacidad es finita.

Un `int` de Java ocupa 32 bits y utiliza representación con signo, por lo que su rango va desde:

-2 147 483 648

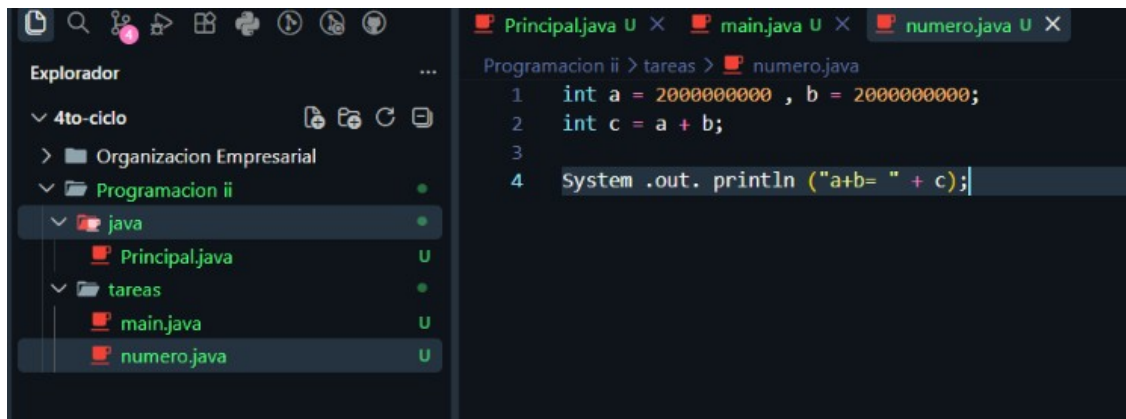
hasta:

2 147 483 647

## 2.5 ¿Por qué se obtiene `a+b = -294967296`?

El código planteado en la clase es:





**Figura 2**

*Código fuente para la demostración de desbordamiento aritmético en Java*

*Nota.* Captura del archivo `numero.java` en el entorno de desarrollo, donde se define la suma de dos variables de tipo `int` que superan el límite superior de representación de 32 bits. Elaboración propia.

Matemáticamente, la suma de los dos valores es:

$$2\,000\,000\,000 + 2\,000\,000\,000 = 4\,000\,000\,000$$

Sin embargo, ese resultado no puede representarse como un valor positivo dentro del rango de un `int`, cuyo máximo es:

$$2\,147\,483\,647$$

Por esta razón ocurre un **overflow** o desbordamiento aritmético.

El resultado matemático:

$$4\,000\,000\,000$$

se encuentra fuera del rango representable por un entero `int` de 32 bits.

Debido a la representación binaria con complemento a dos utilizada para los enteros con signo, los bits resultantes corresponden al valor:

$$-294\,967\,296$$

Por eso Java muestra:

$$a+b= -294967296$$

## 2.5.1 Representación conceptual

El problema puede resumirse de la siguiente manera:

$$2,000,000,000 + 2,000,000,000 = 4,000,000,000$$

Mientras que el rango de `int` es:

$$-2\,147\,483\,648 \leq \text{int} \leq 2\,147\,483\,647$$

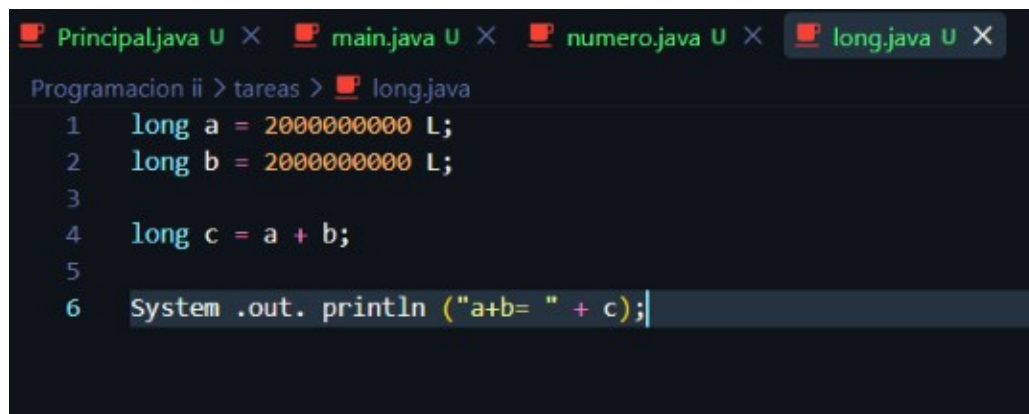
Como:

$$4\,000\,000\,000 > 2\,147\,483\,647$$

el resultado no puede ser almacenado correctamente como un `int` positivo.

## 2.5.2 Solución utilizando `long`

Una forma de evitar el problema es utilizar el tipo `long`, que posee un rango mucho mayor:

A screenshot of an IDE window titled 'long.java'. The code inside is as follows:

```
1 long a = 2000000000L;  
2 long b = 2000000000L;  
3  
4 long c = a + b;  
5  
6 System.out.println("a+b= " + c);
```

The tabs at the top show 'Principal.java', 'main.java', 'numero.java', and 'long.java'. The breadcrumb navigation shows 'Programacion ii > tareas > long.java'.

**Figura 3**

*Solución al desbordamiento mediante el uso del tipo de dato `long`*

*Nota.* Captura del archivo `long.java` en el IDE, donde se declaran variables de tipo `long` con el sufijo `L` para prevenir el desbordamiento numérico y almacenar correctamente la suma. Elaboración propia.

La salida esperada es:

a+b= 4000000000

Es importante agregar la letra `L` a los literales cuando se desea expresar explícitamente que el valor es de tipo `long`.

También debe almacenarse el resultado en una variable `long`.

## 2.6 Relación con el caso de la clase `TV`

El caso `TV` resume varios conceptos iniciales de POO.

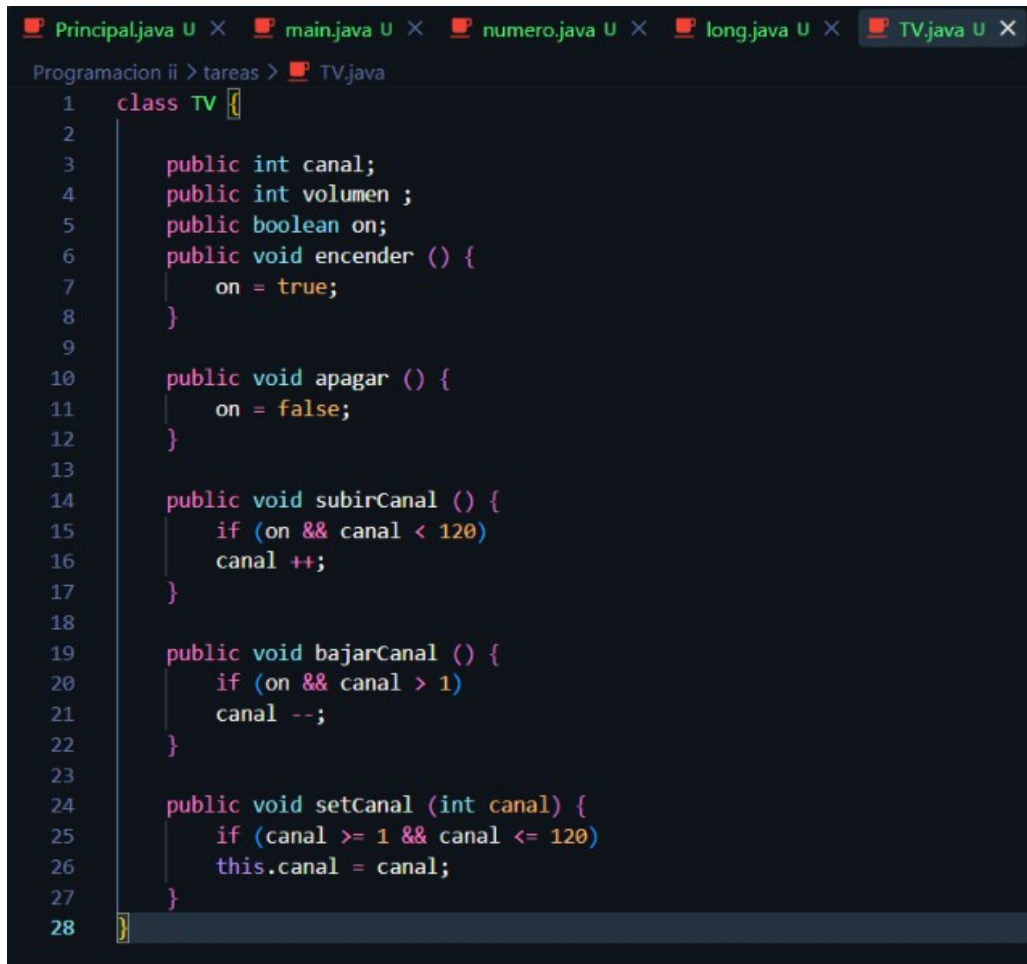
La clase define atributos como `canal`, `volumen` y `on`, mientras que sus métodos representan acciones.

La creación de `tv1` y `tv2` mediante `new TV()` demuestra la creación de instancias.

Además, los métodos `subirCanal()` y `bajarCanal()` contienen condiciones que impiden salir de los límites establecidos para el canal.

El método `setCanal()` valida que el canal se encuentre entre 1 y 120.

Este mecanismo constituye una primera aproximación al **encapsulamiento**, concepto que será profundizado mediante atributos privados, getters y setters.

The image shows a screenshot of a Java IDE with several tabs open: 'Principal.java', 'main.java', 'numero.java', 'long.java', and 'TV.java'. The 'TV.java' tab is active, showing the following code:

```
1  class TV {  
2  
3      public int canal;  
4      public int volumen ;  
5      public boolean on;  
6      public void encender () {  
7          on = true;  
8      }  
9  
10     public void apagar () {  
11         on = false;  
12     }  
13  
14     public void subirCanal () {  
15         if (on && canal < 120)  
16             canal ++;  
17     }  
18  
19     public void bajarCanal () {  
20         if (on && canal > 1)  
21             canal --;  
22     }  
23  
24     public void setCanal (int canal) {  
25         if (canal >= 1 && canal <= 120)  
26             this.canal = canal;  
27     }  
28 }
```

**Figura 4**

*Implementación de la clase TV con atributos, métodos de control y validación de límites*

*Nota.* Captura del archivo TV.java en el entorno de desarrollo, donde se definen los atributos de estado, métodos de encendido/apagado y la lógica condicional para el cambio de canales. Elaboración propia.

## 3 Conclusión

El desarrollo de esta tarea permitió reforzar los conceptos iniciales presentados en la primera clase de Programación 2.

La transición de la programación estructurada hacia la Programación Orientada a Objetos implica una nueva forma de organizar el software, donde las clases permiten definir tipos y los objetos representan instancias concretas con atributos y métodos.

El estudio de las herramientas CASE permitió identificar su utilidad como apoyo a la ingeniería de software, especialmente para modelar, documentar y representar sistemas antes o durante su implementación.

La comparación entre `struct` y `class` mostró que el concepto depende del lenguaje y que, en Java, la clase constituye el mecanismo fundamental para construir tipos orientados a objetos.

Asimismo, la ampliación del método `main()` permitió comprobar experimentalmente el comportamiento de los objetos TV y observar que cada instancia mantiene su propio estado.

Finalmente, el análisis de los tipos de datos de Java permitió comprender que los tipos numéricos poseen límites.

El caso de la suma de dos valores `int` demuestra el fenómeno de overflow y explica por qué una operación matemáticamente positiva puede producir un resultado negativo cuando el valor supera la capacidad del tipo de dato.

En conjunto, estos conceptos constituyen una base necesaria para los siguientes temas del curso, entre ellos la sobrecarga de constructores, `toString()`, encapsulamiento mediante getters y setters, relaciones entre clases, clases abstractas e interfaces.

## 4 Referencias

- Material de clase: *PROGRA 2 - S1 - 27 agosto.txt*, proporcionado para la primera semana del curso.
- Archivo de demostración: *PROGRA2\_DEMOTV.java*, proporcionado para la primera semana del curso.
- Documentación y conocimientos generales sobre Java, UML y herramientas CASE utilizados como complemento explicativo.